

объектно-ориентированное программирование

порождающие паттерны

зачем?

порождающие паттерны

- в общем случае – хотим абстрагировать код, требующий создание объектов, от каких-то деталей этого создания
 - конкретные типы объектов
 - какие-то дополнительные данные
 - источники/способы получения данных
 - внутренняя структура объектов

фабричный метод

проблематика

фабричный метод

- у меня есть логика, в рамках которой создаётся объект
- этот объект может быть разного типа

проблематика (кейс)

фабричный метод

- разрабатываем кассовую систему, необходимо сформировать объект платежа
- платёж может быть нескольких типов
 - разные типы могут иметь разные данные
- для каждого типа есть общая логика

проблематика (кейс)

фабричный метод

```
public record OrderItem(  
    decimal Price,  
    int Amount)  
{  
    public decimal Cost  
        ⇒ Price * Amount;  
}
```

```
public record Order(  
    IEnumerable<OrderItem> Items)  
{  
    public decimal TotalCost  
        ⇒ Items.Sum(x ⇒ x.Cost);  
}
```

проблематика (кейс)

фабричный метод

```
public interface IPayment
{
    decimal Amount { get; }
}
```

```
public record CashPayment(
    decimal Amount)
    : IPayment;
```

```
public record BankPayment(
    decimal Amount,
    string ReceiverAccountId)
    : IPayment;
```

наивная реализация 1`

фабричный метод

```
public class PaymentCalculator(  
    string receiverAccountId)  
{  
    public CashPayment CalculateCashPayment(Order order)  
    {  
        decimal totalCost = order.TotalCost;  
        totalCost = ApplyDiscountsAndCoupons(totalCost);  
  
        return new CashPayment(totalCost);  
    }  
  
    public BankPayment CalculateBankPayment(Order order)  
    {  
        decimal totalCost = order.TotalCost;  
        totalCost = ApplyDiscountsAndCoupons(totalCost);  
  
        return new BankPayment(totalCost, receiverAccountId);  
    }  
  
    private decimal ApplyDiscountsAndCoupons(decimal totalCost)  
    {  
        ...  
    }  
}
```

наивная реализация 1"

фабричный метод

```
public class PaymentCalculator(  
    string receiverAccountId)  
{  
    public IPayment Calculate(Order order, PaymentType type)  
    {  
        decimal totalCost = order.TotalCost;  
        totalCost = ApplyDiscountsAndCoupons(totalCost);  
  
        return type switch  
        {  
            PaymentType.Cash => new CashPayment(totalCost),  
            PaymentType.Bank => new BankPayment(totalCost, receiverAccountId),  
            _ => throw new UnreachableException(),  
        };  
    }  
  
    private decimal ApplyDiscountsAndCoupons(decimal totalCost)  
    {  
        ...  
    }  
}
```

наивная реализация 1*

фабричный метод

- нарушаем SRP и OCP
- один класс будет меняться для обоих типов оплаты
- один класс должен обладать суммарными данными для всех типов оплаты
- вызывающий код должен знать какой тип платежа используется

наивная реализация 2

фабричный метод

```
public interface IPaymentCalculator
{
    IPayment Calculate(Order order);
}
```

наивная реализация 2

фабричный метод

```
public class CashPaymentCalculator : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.TotalCost;
        totalCost = ApplyDiscountsAndCoupons(totalCost);

        return new CashPayment(totalCost);
    }

    private decimal ApplyDiscountsAndCoupons(
        decimal totalCost)
    {
        ...
    }
}
```

```
public class BankPaymentCalculator(
    string receiverAccountId)
    : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.TotalCost;
        totalCost = ApplyDiscountsAndCoupons(totalCost);

        return new BankPayment(totalCost, receiverAccountId);
    }

    private decimal ApplyDiscountsAndCoupons(
        decimal totalCost)
    {
        ...
    }
}
```

наивная реализация 2

фабричный метод

- хорошо
 - не нарушаем SRP и OCP
 - имеем возможность абстрагировать вызывающий код от типа платежа
- плохо
 - публикация логики
 - если изменится логика расчёта скидок, то нужно будет менять её во всех местах

**вариативность создания объектов при
помощи наследования и полиморфизма**

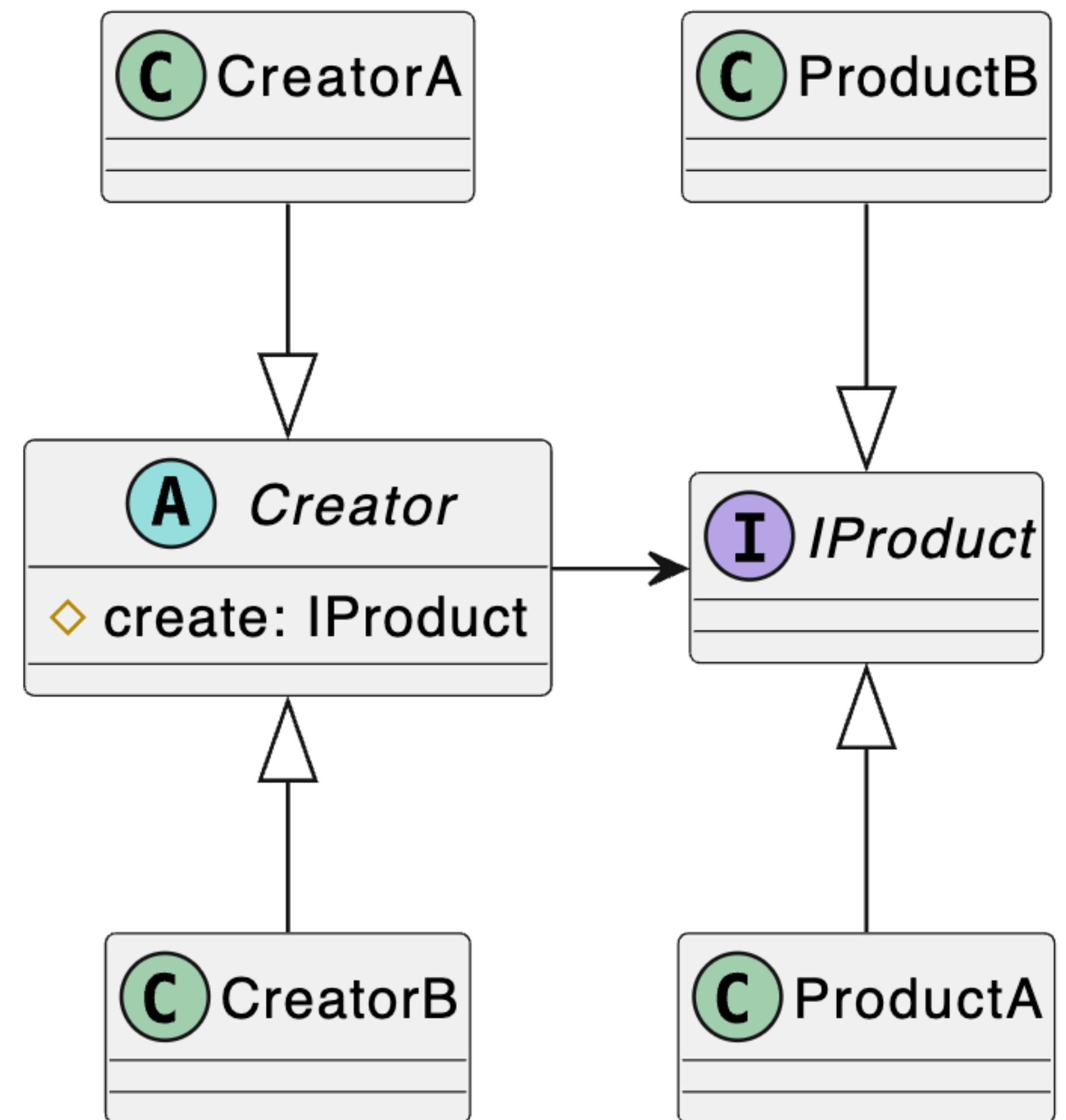


фабричный метод

схема

фабричный метод

- creator
тип, в котором содержится логика, в рамках которой создаются объекты наследники реализуют логику создания объектов
- product
тип, создаваемых объектов наследники создаются в конкретных creator'ах



пример использования фабричный метод

```
public abstract class PaymentCalculatorBase
    : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.TotalCost;
        totalCost = ApplyDiscountsAndCoupons(totalCost);

        return CreatePayment(totalCost);
    }

    protected abstract IPayment CreatePayment(
        decimal totalCost);

    private decimal ApplyDiscountsAndCoupons(
        decimal totalCost)
    {
        ...
    }
}
```

пример использования фабричный метод

```
public abstract class PaymentCalculatorBase
    : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.TotalCost;
        totalCost = ApplyDiscountsAndCoupons(totalCost);

        return CreatePayment(totalCost);
    }

    protected abstract IPayment CreatePayment(
        decimal totalCost);

    private decimal ApplyDiscountsAndCoupons(
        decimal totalCost)
    {
        ...
    }
}
```

пример использования фабричный метод

```
public sealed class CashPaymentCalculator
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new CashPayment(
            totalCost);
    }
}
```

```
public sealed class BankPaymentCalculator(
    string receiverAccountId)
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new BankPayment(
            totalCost,
            receiverAccountId);
    }
}
```

пример использования фабричный метод

```
public sealed class CashPaymentCalculator
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new CashPayment(
            totalCost);
    }
}
```

```
public sealed class BankPaymentCalculator(
    string receiverAccountId)
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new BankPayment(
            totalCost,
            receiverAccountId);
    }
}
```

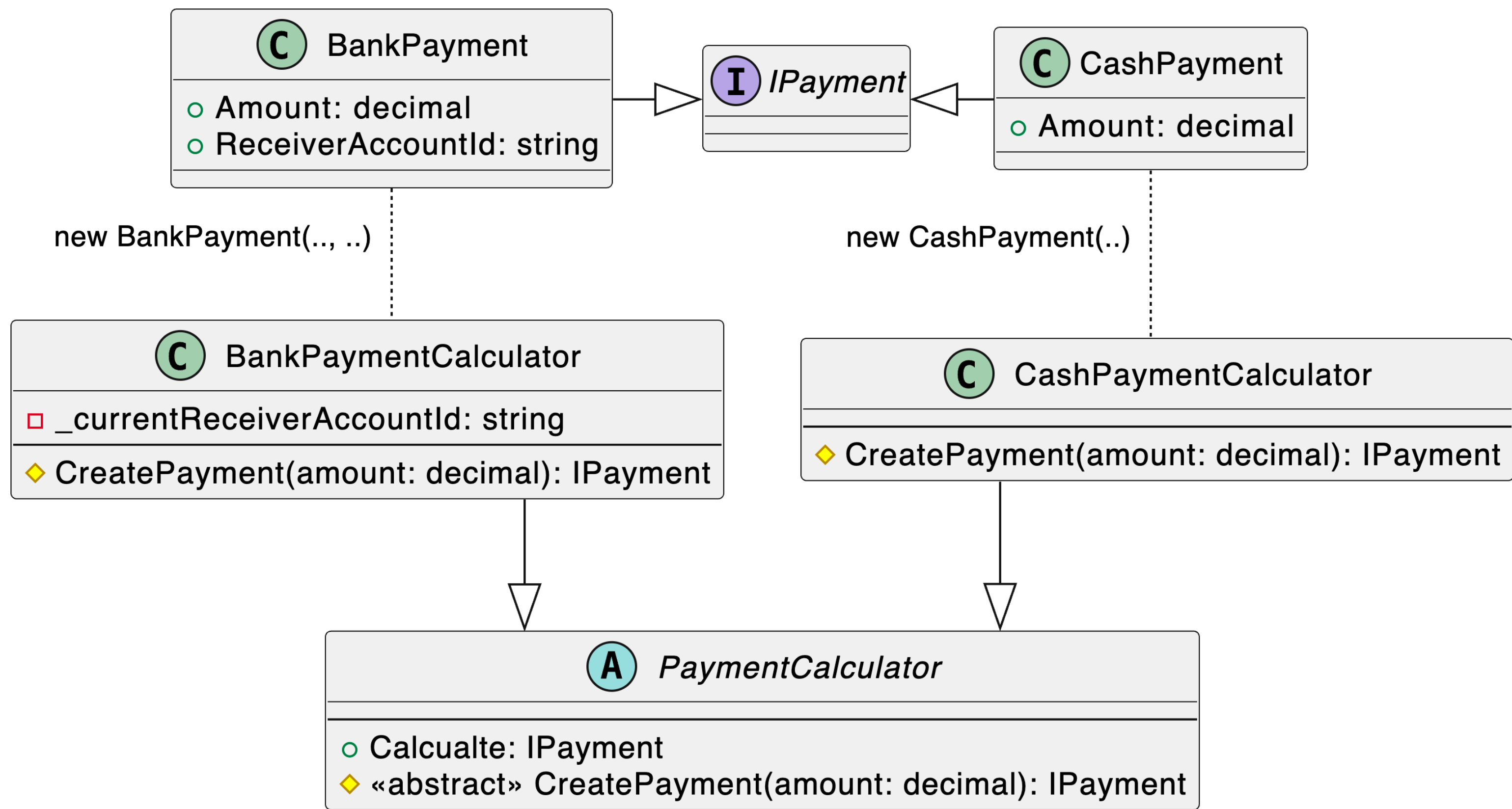
пример использования фабричный метод

```
public sealed class CashPaymentCalculator
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new CashPayment(
            totalCost);
    }
}
```

```
public sealed class BankPaymentCalculator(
    string receiverAccountId)
    : PaymentCalculatorBase
{
    protected override IPayment CreatePayment(
        decimal totalCost)
    {
        return new BankPayment(
            totalCost,
            receiverAccountId);
    }
}
```

схема использования

фабричный метод



преимущества фабричный метод

- в сравнении с наивными реализациями
 - (1*) лучше соблюдаем SRP и OCP
 - (1*) предоставляем возможность абстрагировать вызывающий код от типа платежа (полезно, когда создание платежа происходит в несколько независимых этапов)
 - (2) не дублируем логику

недостатки

фабричный метод

- сильная связанность конкретных создателей с базовым типом
 - логика создания объектов привязана к логике базового типа
 - переиспользование логики создания – копипаста

недостатки фабричный метод

```
public abstract class PaymentCalculator
    : IPaymentCalculator
{
    public IPayment Calculate(Order order) { ... }

    protected abstract IPayment CreatePayment(decimal amount);
}
```

```
public class CashPaymentCalculator
    : PaymentCalculator { ... }
```

```
public class BankPaymentCalculator
    : PaymentCalculator { ... }
```

```
public abstract class FixedPricePaymentCalculator
    : IPaymentCalculator
{
    public IPayment Calculate(Order order) { ... }

    protected abstract IPayment CreatePayment(decimal amount);
}
```

```
public class FixedCashPaymentCalculator
    : FixedPricePaymentCalculator { ... }
```

```
public class FixedBankPaymentCalculator
    : FixedPricePaymentCalculator { ... }
```

недостатки

фабричный метод

- сильная связанность конкретных создателей с базовым типом
 - логика создания объектов привязана к логике базового типа
 - переиспользование логики создания – копипаста
- неявное нарушение SRP
объект конкретного создателя ответственен как за реализацию логики, так и за создание продуктов
 - нельзя протестировать родительскую/дочернюю логику в изоляции

абстрактная фабрика

проблематика

абстрактная фабрика

- всё то же что и в фабричном методе
- + сам фабричный метод

**вариативность создания объектов при
помощи композиции и полиморфизма**

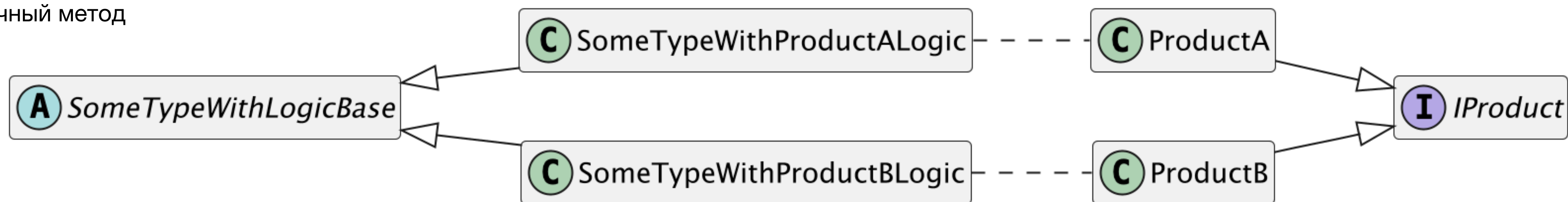


(абстрактная) фабрика

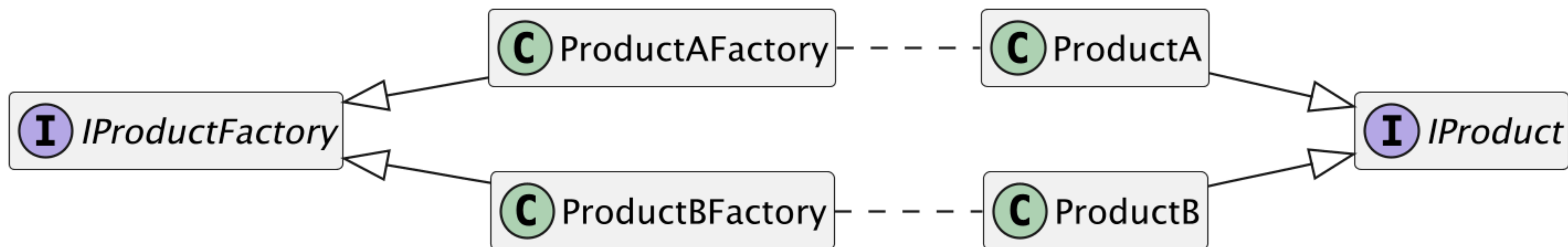
схема

абстрактная фабрика

фабричный метод



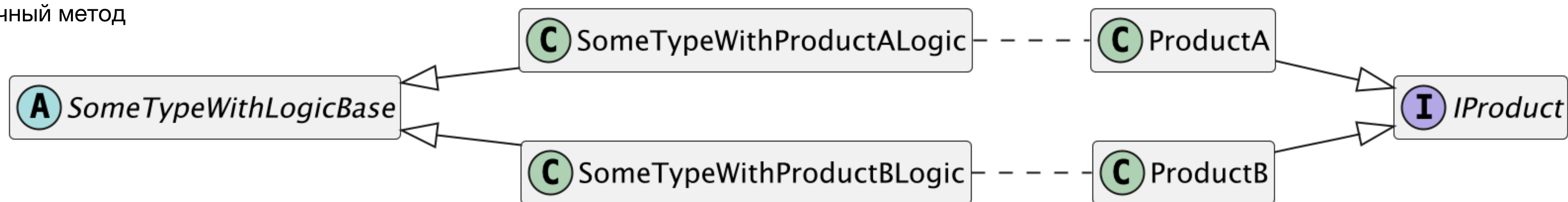
абстрактная фабрика



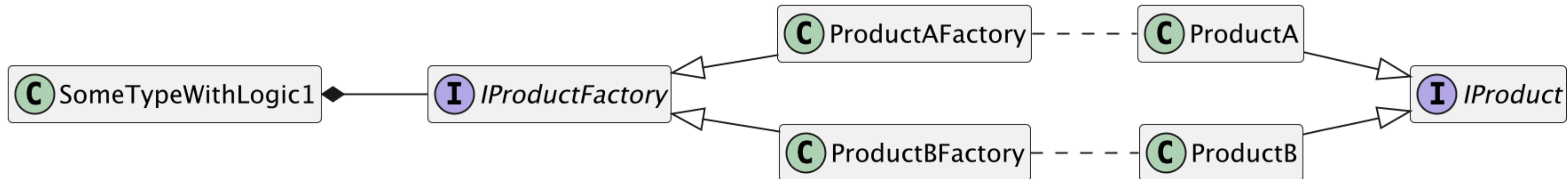
схема

абстрактная фабрика

фабричный метод



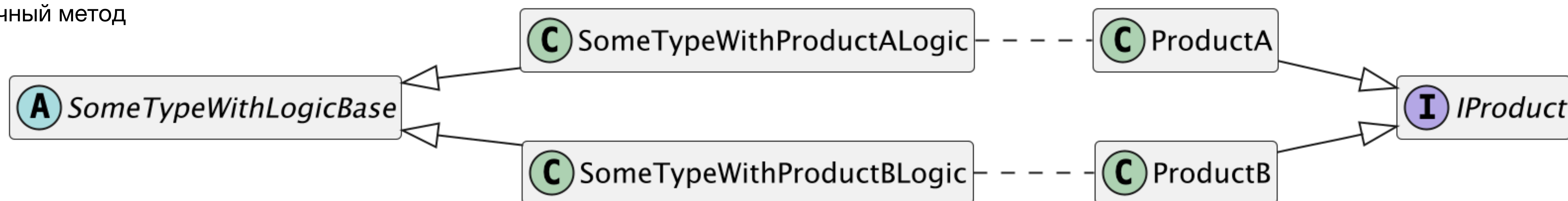
абстрактная фабрика



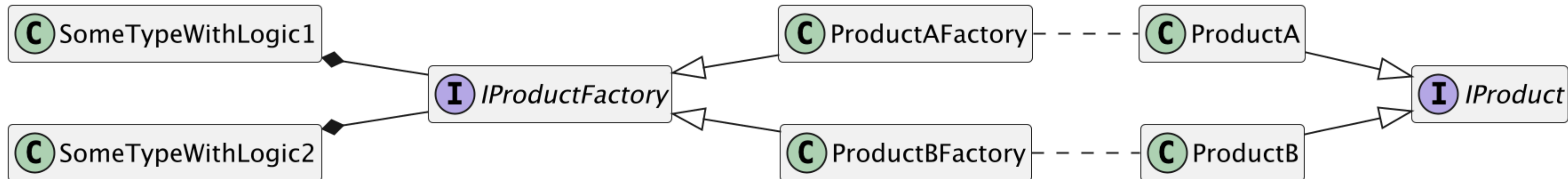
схема

абстрактная фабрика

фабричный метод



абстрактная фабрика



пример использования абстрактная фабрика

```
public interface IPaymentFactory
{
    IPayment Create(decimal amount);
}
```

пример использования абстрактная фабрика

```
public sealed class CashPaymentFactory
    : IPaymentFactory
{
    public IPayment Create(
        decimal amount)
    {
        return new CashPayment(amount);
    }
}
```

```
public sealed class BankPaymentFactory(
    string receiverAccountId)
    : IPaymentFactory
{
    public IPayment Create(decimal amount)
    {
        return new BankPayment(
            amount,
            receiverAccountId);
    }
}
```

пример использования абстрактная фабрика

```
public sealed class PaymentCalculator(
    IPaymentFactory paymentFactory)
    : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.TotalCost;
        totalCost = ApplyDiscountsAndCoupons(totalCost);

        return paymentFactory.Create(totalCost);
    }

    private decimal ApplyDiscountsAndCoupons(
        decimal totalCost)
    {
        ...
    }
}
```

```
public sealed class FixedPaymentCalculator(
    IPaymentFactory paymentFactory,
    decimal fixedPrice)
    : IPaymentCalculator
{
    public IPayment Calculate(Order order)
    {
        decimal totalCost = order.Items
            .Sum(item => item.Amount * fixedPrice);

        return paymentFactory.Create(totalCost);
    }
}
```

преимущества

абстрактная фабрика

- модульность логики создания объектов
 - фабрика – отдельная абстракция, которую можно использовать в любой части программы
- соблюдение SRP на всех уровнях
- возможность изолированного тестирования

сравнение

абстрактная фабрика

- фабричный метод и фабрика решают одни и те же проблемы
- использование фабрики предпочтительно, тк она ведёт к более низкой СВЯЗАННОСТИ
- за исключением случаев когда фабрика наоборот, ведёт к более высокой СВЯЗАННОСТИ
 - если набор входных данных для фабрики сильно связан с ВЫЗЫВАЮЩИМ КОДОМ
 - если гарантируется отсутствие различной логики, вызывающей создание объекта

билдер

проблематика билдер

- создание больших/сложных объектов
 - много атрибутов, значительная часть которых – опциональна или имеет значение по умолчанию
- необходимость промежуточного хранения данных
 - сложная логика сбора данных, множество источников данных

проблематика (кейс)

билдер

```
public sealed record Order(  
    long Id,  
    long CustomerId,  
    IReadOnlyCollection<OrderItem> Items,  
    DateTimeOffset CreatedAt,  
    string Comment);
```

проблематика (кейс)

билдер

```
var order = new Order(  
    Id: 1,  
    CustomerId: 2,  
    Items:  
    [  
        new OrderItem(SkuId: 1, Amount: 2),  
        new OrderItem(SkuId: 3, Amount: 4)  
    ],  
    CreatedAt: DateTimeOffset.UtcNow,  
    Comment: string.Empty);
```

проблематика (кейс)

билдер

- при создании требуется “много” аргументов
- часть из них – значения по-умолчанию
- следовательно, нужно писать много лишнего кода

**выделение отдельного типа,
инкапсулирующего логику сбора данных
и создания объекта**



билдер

параметры и аргументы билдер

параметр

набор тип+имя находящийся в сигнатуре метод

```
public void A(int a, char b);
```

аргумент

значение передающееся в метод

```
obj.A(1, '2');
```

ВИДЫ билдеров

билдер

- convenience builder
 - простой буффер данных, собирает их и создаёт объект
 - предоставляет методы для удобства
 - добавление в коллекции по-элементно
 - аккумуляция числовых значений
 - ИТД...

convenience builder

билдер

```
public class OrderBuilder(  
    long orderId,  
    long customerId)  
{  
    private readonly List<OrderItem> _items = [];  
    private DateTimeOffset _createdAt = DateTimeOffset.UtcNow;  
    private string _comment = string.Empty;  
  
    ...  
}
```

convenience builder

билдер

```
public class OrderBuilder(...)  
{  
    ...  
  
    public OrderBuilder AddItem(OrderItem item)  
    {  
        _items.Add(item);  
        return this;  
    }  
  
    public OrderBuilder AddItems(IEnumerable<OrderItem> items)  
    {  
        _items.AddRange(items);  
        return this;  
    }  
  
    ...  
}
```

convenience builder

билдер

```
public class OrderBuilder(...)  
{  
    ...  
  
    public OrderBuilder CreatedAt(DateTimeOffset createdAt)  
    {  
        _createdAt = createdAt;  
        return this;  
    }  
  
    public OrderBuilder WithComment(string comment)  
    {  
        _comment = comment;  
        return this;  
    }  
  
    ...  
}
```

convenience builder

билдер

```
public class OrderBuilder(...)  
{  
    ...  
  
    public Order Build()  
    {  
        return new Order(  
            orderId,  
            customerId,  
            _items.ToArray(),  
            _createdAt,  
            _comment);  
    }  
}
```

из чего состоит?

билдер

- опционально, входные данные
 - какие-то обязательные параметры для создания объекта
- всегда, набор полей равный набору параметров конструктора объекта
- всегда, методы для задания значений
 - для коллекций: методы добавления одного/множества значений
 - для унарных значений: методы задания (перезаписи), методы агрегации (например сложение)
- всегда (важно!), метод создания объекта
 - опционально, может возвращать результат

convenience builder

билдер

```
var order = new Order(  
    Id: 1,  
    CustomerId: 2,  
    Items:  
    [  
        new OrderItem(SkuId: 1, Amount: 2),  
        new OrderItem(SkuId: 3, Amount: 4)  
    ],  
    CreatedAt: DateTimeOffset.UtcNow,  
    Comment: string.Empty);
```



```
var order = new OrderBuilder(orderId: 1, customerId: 2)  
    .AddItem(new OrderItem(SkuId: 1, Amount: 2))  
    .AddItem(new OrderItem(SkuId: 3, Amount: 4))  
    .Build();
```

convenience builder

билдер

- мы определили буфер данных
- мы определили как его можно наполнять данными
- создание объекта, использование большого конструктора – в отдельном типе

convenience builder

билдер

- не только меньше кода, но ещё и OCP

convenience builder

билдер

```
public class OrderService(  
    int promoItemCount)  
{  
    public Order CreateDefaultOrder(  
        long orderId,  
        long customerId)  
    {  
        return new Order(  
            orderId,  
            customerId,  
            Items: [.. GetPromoItems(), .. GetDiscountedItems()],  
            CreatedAt: DateTimeOffset.UtcNow,  
            Comment: string.Empty);  
    }  
  
    private IEnumerable<OrderItem> GetPromoItems();  
  
    private IEnumerable<OrderItem> GetDiscountedItems();  
}
```

convenience builder

билдер

- нарушаем OCP в примере
 - явно завязываемся на источники данных, формат их возвращаемых значений

convenience builder

билдер

```
public interface IOrderConfigurator
{
    void Configure(OrderBuilder builder);
}
```

```
public class PromoOrderConfigurator(
    int promoItemCount)
    : IOrderConfigurator
{
    public void Configure(OrderBuilder builder);
}
```

```
public class DiscountedItemOrderConfigurator
    : IOrderConfigurator
{
    public void Configure(OrderBuilder builder);
}
```

convenience builder

билдер

```
public class OrderService(  
    IEnumerable<IOrderConfigurator> configurators)  
{  
    public Order CreateDefaultOrder(  
        long orderId,  
        long customerId)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        foreach (IOrderConfigurator configurator in configurators)  
        {  
            configurator.Configure(builder);  
        }  
  
        return builder.Build();  
    }  
}
```

convenience builder

билдер

- соблюдаем OCP в примере
 - источники данных – абстрактные
 - мы ничего не знаем об их количестве
 - можем добавлять новые

convenience builder

билдер

- текущая реализация не практична
 - заказ создаётся только с товарами по-умолчанию
 - нет возможности добавить больше товаров

convenience builder

билдер

```
public class OrderService(  
    IEnumerable<IOrderConfigurator> configurators)  
{  
    public Order CreateDefaultOrder(  
        long orderId,  
        long customerId)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        foreach (IOrderConfigurator configurator in configurators)  
        {  
            configurator.Configure(builder);  
        }  
  
        return builder.Build();  
    }  
}
```

convenience builder

билдер

```
public class OrderService(  
    IEnumerable<IOrderConfigurator> configurators)  
{  
    public Order CreateDefaultOrder(  
        long orderId,  
        long customerId,  
        IEnumerable<OrderItem> orderItems)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        foreach (IOrderConfigurator configurator in configurators)  
        {  
            configurator.Configure(builder);  
        }  
  
        builder.AddItem(orderItems);  
  
        return builder.Build();  
    }  
}
```

convenience builder

билдер

- нарушаем OCP
 - понадобятся новые параметры – нужно менять код сервиса
- нарушаем DIP
 - параметры метода зависят от его потребителей
 - если у заказа появятся дополнительные параметры, их также придётся передавать

convenience builder

билдер

```
public class OrderService(  
    IEnumerable<IOrderConfigurator> configurators)  
{  
    public OrderBuilder CreateDefaultBuilder(  
        long orderId,  
        long customerId)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        foreach (IOrderConfigurator configurator in configurators)  
        {  
            configurator.Configure(builder);  
        }  
  
        return builder;  
    }  
}
```

convenience builder

билдер

- проблема – теряем возможность валидаций
 - допустим есть максимальный вес заказа

ВИДЫ билдеров

билдер

- stateful constructor builder
 - “если бы конструктор был объектом”, когда логика создания объекта превосходит возможности конструктора
 - эта логика всё равно где-то будет, такой билдер – единая входная точка
- инкапсулирует бизнес логику
 - сложные агрегации данных
 - валидации

stateful constructor builder

билдер

```
public class OrderBuilder(  
    long orderId,  
    long customerId,  
    double maxWeight)  
{  
    ...  
}
```

stateful constructor builder

билдер

```
public class OrderBuilder(  
    ...  
    double maxWeight)  
{  
    ...  
  
    public BuildOrderResult Build()  
    {  
        double totalWeight = _items.Sum(item => item.SingleItemWeight * item.Amount);  
  
        if (totalWeight > maxWeight)  
            return new BuildOrderResult.MaxWeightExceeded();  
  
        var order = new Order(  
            orderId,  
            customerId,  
            _items.ToArray(),  
            _createdAt,  
            _comment,  
            totalWeight);  
  
        return new BuildOrderResult.Success(order);  
    }  
}
```

stateful constructor builder

билдер

- такой билдер – больше чем буфер данных
- его назначение не просто собрать и создать, он является полноценной абстракцией
 - может иметь своё поведение
 - может знать об инвариантах модели
 - может содержать другие абстракции

stateful constructor builder

билдер

```
public class OrderBuilder(...)
{
    ...

    private double _totalWeight;

    public bool CanAddItem(OrderItem item)
    {
        return _totalWeight + item.TotalWeight < maxWeight;
    }

    public OrderBuilder AddItem(OrderItem item)
    {
        if (CanAddItem(item) is false)
        {
            throw new InvalidOperationException(
                "Adding this item will exceed weight limitations");
        }

        _items.Add(item);
        _totalWeight += item.TotalWeight;

        return this;
    }

    ...
}
```

преимущества билдер

- происходит “инверсия зависимостей”
 - не логика создания получает все данные
 - а источники данные получают возможность работы с логикой создания

преимущества билдер

- снижается связанность
 - нет необходимости прописывать все возвращаемые данные в сигнатурах
 - нет необходимости выделять типы для промежуточного хранения данных
 - источник данных решает какие данные задавать, меньше вероятность забыть какие-то данные при расширении контракта

преимущества билдер

```
public sealed record DefaultOrderArgs(  
    string DefaultComment,  
    IEnumerable<OrderItem> DefaultOrderItems);
```

```
public class OrderService(  
    DefaultOrderArgsProvider provider)  
{  
    public Order CreateDefaultBuilder(  
        long orderId,  
        long customerId)  
    {  
        DefaultOrderArgs defaultArgs = provider.Create();  
  
        return new Order(  
            Id: 1,  
            CustomerId: 2,  
            Items: defaultArgs.DefaultOrderItems.ToArray(),  
            Comment: defaultArgs.DefaultComment,  
            CreatedAt: DateTimeOffset.UtcNow,  
            TotalWeight: ...);  
    }  
}
```

```
public class OrderService(  
    IOrderConfigurator configurator)  
{  
    public Order CreateDefaultBuilder(  
        long orderId,  
        long customerId)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        configurator.Configure(builder);  
  
        return builder.Build();  
    }  
}
```

преимущества билдер

```
public sealed record DefaultOrderArgs(  
    string DefaultComment,  
    IEnumerable<OrderItem> DefaultOrderItems,  
    double PackageWeight);
```

```
public class OrderService(  
    DefaultOrderArgsProvider provider)  
{  
    public Order CreateDefaultBuilder(  
        long orderId,  
        long customerId)  
    {  
        DefaultOrderArgs defaultArgs = provider.Create();  
  
        return new Order(  
            Id: 1,  
            CustomerId: 2,  
            Items: defaultArgs.DefaultOrderItems.ToArray(),  
            Comment: defaultArgs.DefaultComment,  
            CreatedAt: DateTimeOffset.UtcNow,  
            TotalWeight: ...+ defaultArgs.PackageWeight);  
    }  
}
```

```
public class OrderService(  
    IOrderConfigurator configurator)  
{  
    public Order CreateDefaultBuilder(  
        long orderId,  
        long customerId)  
    {  
        var builder = new OrderBuilder(orderId, customerId);  
  
        configurator.Configure(builder);  
  
        return builder.Build();  
    }  
}
```

преимущества билдер

- повышается прослеживаемость ошибок
 - при сборе данных, провалидировать их можно только в самом конце
 - при наполнении билдера, валидация может выполняться в процессе
 - ошибка возникает рядом с источником данных, а не при создании объекта

преимущества билдер

```
var orderBuilder = new OrderBuilder()  
    .AddItem(new OrderItem(...));
```



```
AddDefaultItems(orderBuilder);  
AddRequestedItems(orderBuilder);  
AddForecastedItems(orderBuilder);
```

```
Order order = orderBuilder.Build();
```

полиморфизм

билдер

```
public interface IOrderBuilder
{
    IOrderBuilder WithItem(OrderItem item);

    Order Build();
}
```

```
public class UnlimitedOrderBuilder : IOrderBuilder { ... }
```

```
public class LimitedOrderBuilder : IOrderBuilder { ... }
```

директор билдер

```
public record Pizza(  
    PizzaSize Size,  
    DoughType DoughType,  
    Sause Sause,  
    IReadOnlyCollection<Topping> Toppings);
```

```
public class PizzaBuilder  
{  
    private readonly List<Topping> _toppings = [];  
    private PizzaSize _size = PizzaSize.Medium;  
    private DoughType _doughType = DoughType.Standard;  
    private Sause _sause = Sause.Tomato;  
  
    public PizzaBuilder WithTopping(Topping topping) { ... }  
  
    public PizzaBuilder WithSize(PizzaSize size) { ... }  
  
    public PizzaBuilder WithDoughType(DoughType type) { ... }  
  
    public PizzaBuilder WithSause(Sause sause) { ... }  
  
    public Pizza Build() { ... }  
}
```

директор билдер

```
public interface IPizzaDirector
{
    PizzaBuilder Direct(PizzaBuilder builder);
}

public class PepperoniPizzaDirector : IPizzaDirector
{
    public PizzaBuilder Direct(PizzaBuilder builder)
    {
        return builder
            .WithDoughType(DoughType.Standard)
            .WithSause(Sause.Tomato)
            .WithSize(PizzaSize.Medium)
            .WithTopping(Topping.Cheese)
            .WithTopping(Topping.Pepperoni);
    }
}
```

```
var pizzaBuilder = new PizzaBuilder();
var pepperoniDirector = new PepperoniPizzaDirector();

var myPizza = pepperoniDirector
    .Direct(pizzaBuilder)
    .WithTopping(Topping.Jalapeno)
    .WithSize(PizzaSize.Large)
    .Build();
```

директор

билдер

```
public static class PizzaBuilderExtensions
{
    public static PizzaBuilder DirectPepperoni(
        this PizzaBuilder builder)
    {
        return builder
            .WithDoughType(DoughType.Standard)
            .WithSause(Sause.Tomato)
            .WithSize(PizzaSize.Medium)
            .WithTopping(Topping.Cheese)
            .WithTopping(Topping.Pepperoni);
    }
}
```

```
var myPizza = new PizzaBuilder()
    .DirectPepperoni()
    .WithTopping(Topping.Jalapeno)
    .WithSize(PizzaSize.Large)
    .Build();
```

interface driven билдер

```
public record Email(  
    string Address,  
    string Subject,  
    string Body);
```

```
public class EmailBuilder  
{  
    private string? _address;  
    private string _subject = string.Empty;  
    private string _body = string.Empty;  
  
    public EmailBuilder WithAddress(string address) { }  
  
    public EmailBuilder WithSubject(string subject) { }  
  
    public EmailBuilder WithBody(string body) { }  
  
    public Email Build()  
    {  
        if (_address is null)  
            throw new ArgumentNullException();  
  
        return new Email(_address, _subject, _body);  
    }  
}
```

interface driven билдер



```
var email = new EmailBuilder()  
    .WithBody("Hello!")  
    .Build();
```



builder

interface driven

```
public interface IEmailAddressBuilder
{
    IEmailBuilder WithAddress(string address);
}

public interface IEmailBuilder
{
    IEmailBuilder WithSubject(string subject);
    IEmailBuilder WithBody(string body);
    Email Build();
}
```

```
public class Email
{
    public required string Address { get; init; }
    public required string Subject { get; init; }
    public required string Body { get; init; }

    public static IEmailAddressBuilder Builder
        => new EmailBuilder();

    private class EmailBuilder
        : IEmailAddressBuilder, IEmailBuilder
    {
        public IEmailBuilder WithAddress(string address) { ... }

        public IEmailBuilder WithSubject(string subject) { ... }

        public IEmailBuilder WithBody(string body) { ... }

        public Email Build() { ... }
    }
}
```

builder

interface driven



```
var email = Email.Builder  
    .WithBody("Hello!")  
    .Build();
```

прототип

проблематика

прототип

- необходимо копировать объекты со сложной структурой

кейс

прототип

- реализация системы учета git репозиториев

кейс

прототип

```
public sealed record Commit(string Hash, string Message, string? ParentHash);  
  
public sealed record Branch(string Name, string HeadHash);
```

кейс

прототип

```
public sealed class BranchCommit(  
    string hash,  
    string message,  
    BranchCommit? parent)  
{  
    public string Hash { get; } = hash;  
  
    public string Message { get; private set; } = message;  
  
    public BranchCommit? Parent { get; private set; } = parent;  
  
    ...  
}
```

кейс

прототип

```
public sealed class BranchCommit(...)
{
    ...

    public void Ammend(string newMessage)
    {
        Message = newMessage;
    }

    public bool HasCommit(string hash)
    {
        if (Hash == hash)
            return true;

        return Parent?.HasCommit(hash) ?? false;
    }

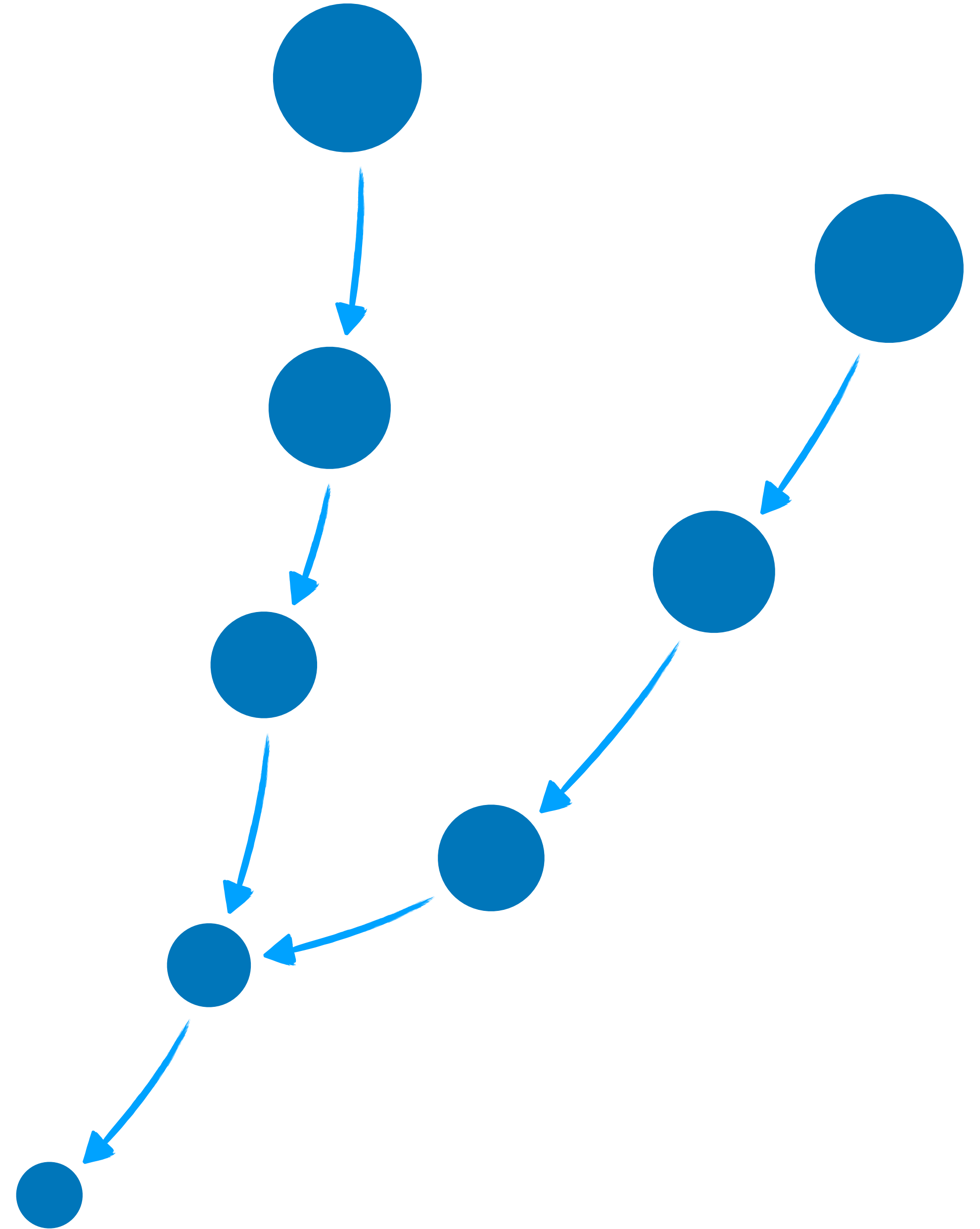
    ...
}
```

кейс прототип

```
public sealed class BranchCommit(...)
{
    ...

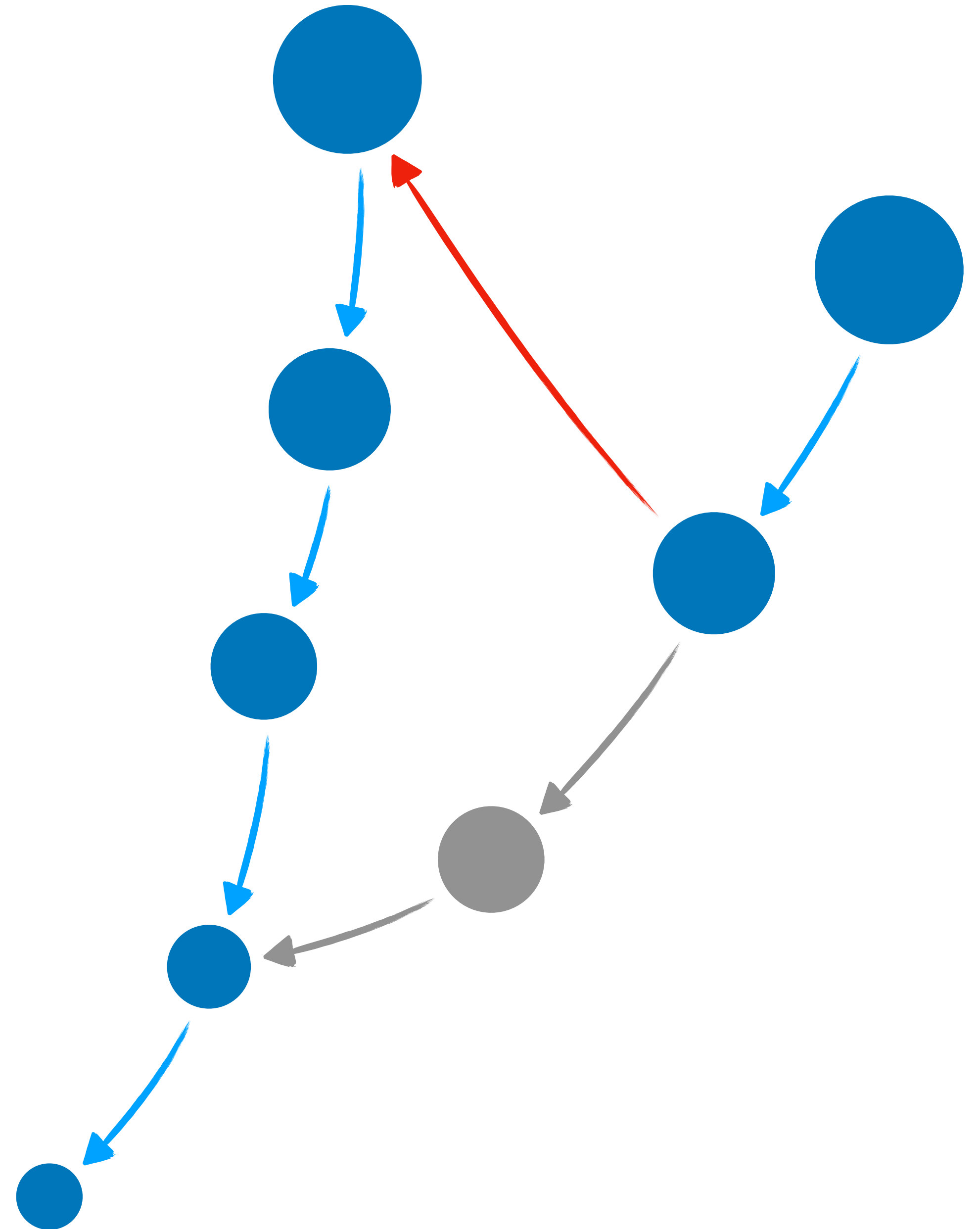
    public void Rebase(BranchCommit newParent)
    {
        if (Parent is not null && newParent.HasCommit(Parent.Hash) is false)
        {
            throw new InvalidOperationException(
                "Rebasing not from branching point");
        }

        Parent = newParent;
    }
}
```



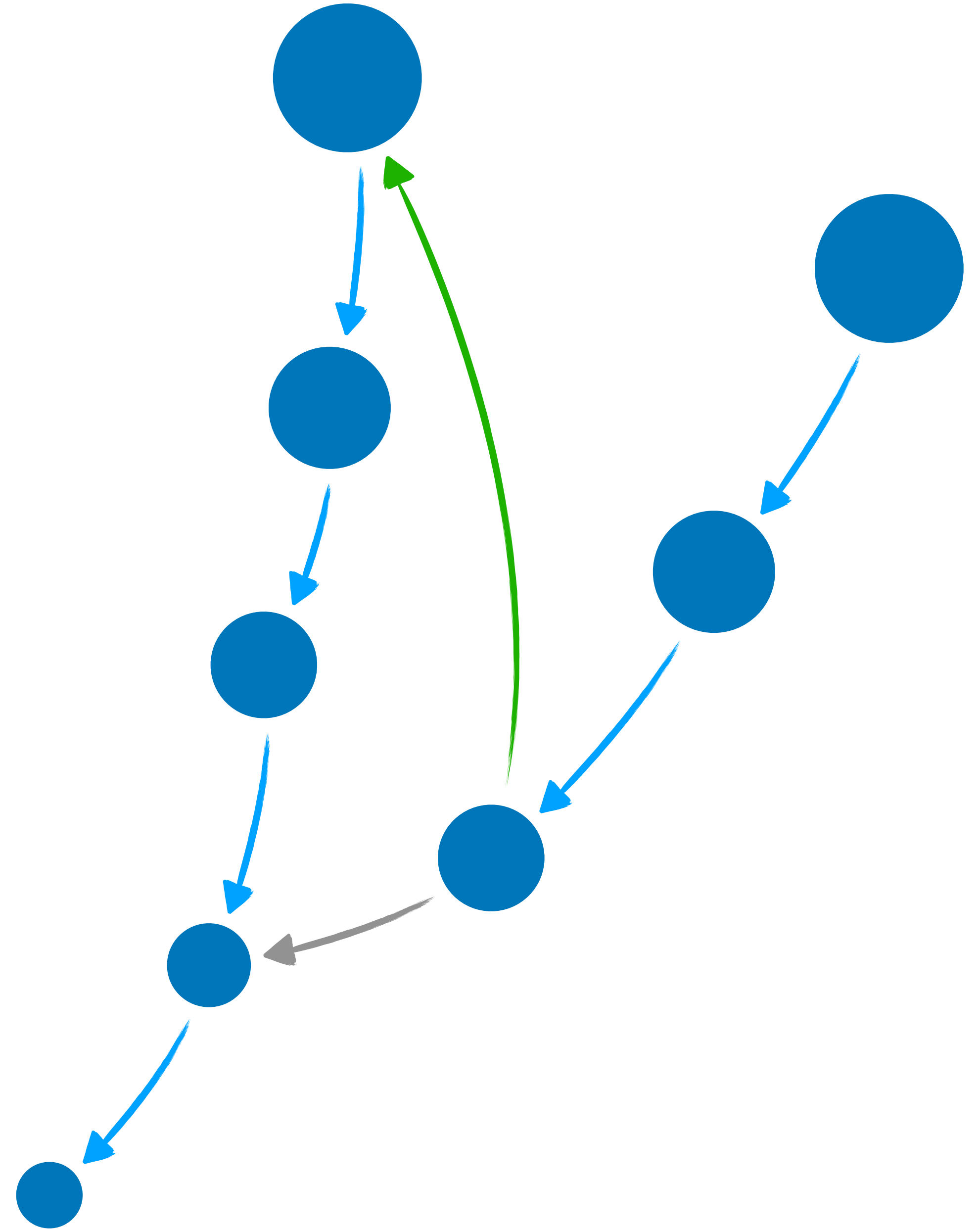
кейс прототип

```
public sealed class BranchCommit(...)  
{  
    ...  
    public void Rebase(BranchCommit newParent)  
    {  
        if (Parent is not null && newParent.HasCommit(Parent.Hash) is false)  
        {  
            throw new InvalidOperationException(  
                "Rebasing not from branching point");  
        }  
        Parent = newParent;  
    }  
}
```



кейс прототип

```
public sealed class BranchCommit(...)  
{  
    ...  
    public void Rebase(BranchCommit newParent)  
    {  
        if (Parent is not null && newParent.HasCommit(Parent.Hash) is false)  
        {  
            throw new InvalidOperationException(  
                "Rebasing not from branching point");  
        }  
        Parent = newParent;  
    }  
}
```



кейс

прототип

```
public sealed class Repository
{
    private readonly Dictionary<string, BranchCommit> _branches = [];

    public Repository(
        IEnumerable<Branch> branches,
        IEnumerable<Commit> commits)
    {
        ...
    }
}
```

кейс

прототип

```
public sealed class Repository
{
    ...

    public Repository(...)
    {
        var commitsDictionary = commits.ToDictionary(commit => commit.Hash);

        _branches = branches.ToDictionary(
            branch => branch.Name,
            branch => CreateBranchCommit(branch.HeadHash));

        BranchCommit CreateBranchCommit(string hash)
        {
            Commit commit = commitsDictionary[hash];

            return new BranchCommit(
                commit.Hash,
                commit.Message,
                commit.ParentHash is null ? null : CreateBranchCommit(commit.ParentHash));
        }
    }

    ...
}
```

кейс

прототип

```
public sealed class Repository
{
    ...

    public IEnumerable<string> BranchNames => _branches.Keys;

    public IEnumerable<Commit> EnumerateBranch(string branchName)
    {
        if (_branches.TryGetValue(branchName, out BranchCommit? commit) is false)
        {
            throw new InvalidOperationException("Branch not found");
        }

        var commits = new List<Commit>();

        while (commit is not null)
        {
            commits.Add(new Commit(commit.Hash, commit.Message, commit.Parent?.Hash));
            commit = commit.Parent;
        }

        return commits;
    }
}
```

кейс

прототип

- что делают с git репозиториями?

кейс

прототип

- что делают с git репозиториями? клонируют.

наивная реализация

прототип

```
Branch[] branches = repository.BranchNames
    .Select(branchName => new Branch(
        branchName,
        repository.EnumerateBranch(branchName).First().Hash))
    .ToArray();

Commit[] allCommits = repository.BranchNames
    .SelectMany(branchName => repository.EnumerateBranch(branchName))
    .ToArray();

return new Repository(branches, allCommits);
```

наивная реализация

прототип

```
Repository Clone(Repository repository)
{
    Branch[] branches = repository.BranchNames
        .Select(branchName => new Branch(
            branchName,
            repository.EnumerateBranch(branchName).First().Hash))
        .ToArray();

    Commit[] allCommits = repository.BranchNames
        .SelectMany(branchName => repository.EnumerateBranch(branchName))
        .ToArray();

    return new Repository(branches, allCommits);
}
```

наивная реализация

прототип

- хранимые данные могут отличаться от параметров конструктора
- хранимые данные могут быть приватными
- конвертация данных может быть дорогой
 - может быть невозможна (не все данные доступны извне)
- дубликация кода

инкапсуляция логики копирования объекта



прототип

ВИДЫ КОПИРОВАНИЯ

прототип

- shallow copy
 - копируется только верхнеуровневый объект
 - вложенные объекты остаются теми же
- deep copy
 - копируются все объекты по дереву ссылок
- комбинированный
 - копируется какая-то часть дерева объектов
- **!** есть смысл копировать только изменяемые объекты

shallow copy

прототип

```
public sealed class Repository
{
    ...

    private Repository(Dictionary<string, BranchCommit> branches)
    {
        _branches = branches;
    }

    ...

    public Repository Clone()
    {
        return new Repository(_branches.ToDictionary(kvp => kvp.Key, kvp => kvp.Value));
    }
}
```

shallow copy

прототип

- скопировали только объект репозитория и словарь
- добавление ветки в новый репозиторий не затронет старый
- но изменение коммита в новом репозитории – затронет

shallow copy

прототип

```
public sealed class Repository
{
    ...

    public BranchCommit GetBranchCommit(string branchName, string hash)
    {
        ...
    }

    ...
}
```

shallow copy

прототип

```
var first = new Repository(...);
Repository second = first.Clone();

Console.WriteLine(first.GetBranchCommit("master", "abc").Message);    old message

BranchCommit commit = second.GetBranchCommit("master", "abc");
commit.Ammend("New message");

Console.WriteLine(first.GetBranchCommit("master", "abc").Message);    New message
Console.WriteLine(second.GetBranchCommit("master", "abc").Message);    New message
```

deep copy

прототип

```
public sealed class BranchCommit(...)
{
    ...

    public BranchCommit Clone()
    {
        return new BranchCommit(
            Hash,
            Message,
            Parent?.Clone());
    }
}
```

```
public sealed class Repository
{
    ...

    public Repository Clone()
    {
        return new Repository(_branches.ToDictionary(
            kvp => kvp.Key,
            kvp => kvp.Value.Clone()));
    }
}
```

deep copy

прототип

- скопировали и словарь и все объекты внутри него
- деревья объектов абсолютно разные в старом и новом репозитории

deep copy

прототип

```
var first = new Repository(...);
Repository second = first.Clone();

Console.WriteLine(first.GetBranchCommit("master", "abc").Message);    old message

BranchCommit commit = second.GetBranchCommit("master", "abc");
commit.Ammend("New message");

Console.WriteLine(first.GetBranchCommit("master", "abc").Message);    old message
Console.WriteLine(second.GetBranchCommit("master", "abc").Message);    New message
```

почему не просто конструктор?

прототип

- логика копирования может быть необходима в нескольких местах
- данные могут быть сокрыты, модифицированы в конструкторе
- объект находится в иерархии, при копировании конкретный тип не известен

полиморфизм

прототип

```
public interface IShape
{
    void MoveBy(int x, int y);
}
```

полиморфизм

прототип

```
public class Circle(  
    Position position,  
    int radius)  
    : IShape  
{  
    public Position Position { get; private set; } = position;  
  
    public int Radius { get; private set; } = radius;  
  
    public void MoveBy(int x, int y)  
    {  
        Position = Position.MovedBy(x, y);  
    }  
  
    public void Stretch(double factor)  
    {  
        Radius = (int)(Radius * factor);  
    }  
}
```

```
public class Square(  
    Position position,  
    int length)  
    : IShape  
{  
    public Position Position { get; private set; } = position;  
  
    public int Length { get; private set; } = length;  
  
    public void MoveBy(int x, int y)  
    {  
        Position = Position.MovedBy(x, y);  
    }  
}
```

наивная реализация

прототип

```
IShape Clone(IShape shape)
{
    return shape switch
    {
        Circle circle ⇒ new Circle(circle.Position, circle.Radius),
        Square square ⇒ new Square(square.Position, square.Length),
        _ ⇒ throw new ArgumentOutOfRangeException(nameof(shape)),
    };
}
```

наивная реализация

прототип

- нарушаем OCP
- интроспекция – плохо

полиморфизм

прототип

```
public interface IShape
{
    void MoveBy(int x, int y);

    IShape Clone();
}
```

полиморфизм

прототип

```
public class Circle(...)
    : IShape
{
    ...

    public IShape Clone()
    {
        return new Circle(Position, Radius);
    }
}
```

```
public class Square(...)
    : IShape
{
    ...

    public IShape Clone()
    {
        return new Square(Position, Length);
    }
}
```

СИНГЛТОН

проблематика

СИНГЛТОН

- нужно гарантировать что объект определённого типа будет создан всего один раз

реализация

СИНГЛТОН

```
public class Singleton
{
    private static readonly object _lock = new();
    private static Singleton? _instance;

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            if (_instance is not null)
                return _instance;

            lock (_lock)
            {
                if (_instance is not null)
                    return _instance;

                return _instance = new Singleton();
            }
        }
    }
}
```

lazy

СИНГЛТОН

```
public class Singleton
{
    private static readonly Lazy<Singleton> _instance;

    static Singleton()
    {
        _instance = new Lazy<Singleton>(() => new Singleton(), LazyThreadSafetyMode.ExecutionAndPublication);
    }

    private Singleton() { }

    public static Singleton Instance => _instance.Value;
}
```



СИНГЛТОН

- None
не гарантируется потокобезопасность, при инициализации несколькими потоками, объект будет создан несколько раз, сохранённое значение не определено
- PublicationOnly
при инициализации несколькими потоками, объект будет создан несколько раз, сохранённое значение – созданное последним потоком начавшим инициализацию
- ExecutionAndPublication
полная потокобезопасность, при инициализации несколькими потоками, объект будет создан лишь один раз

недостатки

СИНГЛТОН

- тестирование
приватный конструктор не даёт возможности контролировать объект в тестах
- внедрение зависимостей
приватный конструктор не даёт возможности передавать значения извне
- время жизни объекта
т.к. объект инициализируется статически, его время жизни нельзя явно контролировать
- статический стейт
объект можно получить из любого места приложения, без какого-либо контроля

применимость

синглтон

- инфраструктурный код
 - статический контекст, когда нужно передавать информацию прозрачно для пользователя
- DI контейнеры
 - библиотеки, реализующие абстрактные фабрики
 - сам тип не обязательно “каноничный” синглтон, но он будет в единственном экземпляре в рамках одной фабрики

прототип*

проблематика при полиморфизме

прототип*

- у полиморфных прототипов – вымывается тип

```
public class Circle(...)
    : IShape
{
    ...

    public IShape Clone()
    {
        return new Circle(Position, Radius);
    }
}
```



```
public class Square(...)
    : IShape
{
    ...

    public IShape Clone()
    {
        return new Square(Position, Length);
    }
}
```



проблематика при полиморфизме

прототип*

```
var circle = new Circle(new Position(1, 1), radius: 2);
```

```
IShape copy = circle.Clone();  
copy.Stretch(1.2);
```

ТИПИЗАЦИЯ

прототип*

- задача – получать копию с конкретным типом

ТИПИЗАЦИЯ через наследование

прототип*

- при наследовании, компилятор позволяет конкретизировать возвращаемый тип

```
public abstract class Shape
{
    public abstract Shape Clone();
}
```

```
public sealed class Rectangle : Shape
{
    public override Rectangle Clone()
    {
        return new Rectangle();
    }
}
```

ТИПИЗАЦИЯ через реализацию

прототип*

- в C# существует понятие “явной реализации”/“explicit implementation”
- при такой реализации, метод можно вызвать только если работать с объектом через интерфейс

```
public interface ISomeInterface
{
    void Do();
}
```

```
public class SomeType : ISomeInterface
{
    void ISomeInterface.Do() { }

    public void MyDo() { }
}
```

ТИПИЗАЦИЯ через реализацию

прототип*

```
var someObject = new SomeType();  
someObject.MyDo();  
someObject.Do();
```

```
ISomeInterface someInterfaceObject = someObject;  
someInterfaceObject.Do();
```

ТИПИЗАЦИЯ через реализацию

прототип*

```
public class Circle(...)
    : IShape
{
    ...

    public Circle Clone()
    {
        return new Circle(Position, Radius);
    }

    IShape IShape.Clone()
    {
        return Clone();
    }
}
```

`var circle = new Circle(new Position(1, 1), radius: 2);`

`Circle copy = circle.Clone();`
`copy.Stretch(1.2);`

`IShape untypedCopy = ((IShape)circle).Clone();`
`untypedCopy.Stretch(1.2);`

кейс

прототип*

- зачастую в графических редакторах есть функционал дубликации элемента
- при дубликации элемент сдвигается, так пользователь видит что копия произошла
- для этого можно выделить общую логику
 - но тут ломается типизация...

кейс

прототип*

```
IShape DuplicateShape(IShape shape)
{
    IShape copy = shape.Clone();
    copy.MoveBy(1, 1);

    return copy;
}
```

```
var circle = new Circle(new Position(1, 1), radius: 2);

IShape copiedCircle = DuplicateShape(circle);
copiedCircle.Stretch(1.2);
```

кейс

прототип*

```
IShape DuplicateShape(IShape shape)
{
    IShape copy = shape.Clone();
    copy.MoveBy(1, 1);

    return copy;
}
```

```
var circle = new Circle(new Position(1, 1), radius: 2);

Circle copiedCircle = (Circle)DuplicateShape(circle);
copiedCircle.Stretch(1.2);
```

кейс

прототип*

```
IShape DuplicateShape(IShape shape)
{
    IShape copy = shape.Clone();
    copy = new MovedShapeDecorator(copy, 1, 1);

    return copy;
}
```

```
var circle = new Circle(new Position(1, 1), radius: 2);

Circle copiedCircle = (Circle)DuplicateShape(circle);
copiedCircle.Stretch(1.2);
```



параметр-тип,
ссылающийся на себя в ограничениях,
наложенных на допустимые аргументы типы



**рекурсивный параметр-тип
или рекурсивный дженерик**

рекурсивные параметры-типы

прототип*

- обычно встречается в интерфейсах, которые подразумевают, что переданный аргумент-тип, будет тот же что и тип, реализующий интерфейс
- ещё без рекурсии, но уже близко...

```
public interface IEquatable<T>
{
    bool Equals(T? other);
}
```

```
public class UserId(long value) : IEquatable<UserId>
{
    public long Value { get; } = value;

    public bool Equals(UserId? other)
    {
        return other is not null
            && other.Value.Equals(Value);
    }
}
```

рекурсивные параметры-типы

прототип*

```
bool ContainsElement<T>(T element, IEnumerable<T> values)
    where T : IEquatable<T>
{
    foreach (T value in values)
    {
        if (value.Equals(element))
            return true;
    }

    return false;
}
```

```
public class DistinctCollection<T>
    where T : IEquatable<T>
{
    private readonly List<T> _values = [];

    public bool TryAdd(T element)
    {
        if (ContainsElement(element, _values))
            return false;

        _values.Add(element);
        return true;
    }
}
```

рекурсивные параметры-типы

прототип*

```
var collection = new DistinctCollection<UserId>();  
var badCollection = new DistinctCollection<NonEquatableType>();
```



рекурсивные параметры-типы

прототип*

```
public interface IShape
{
    void MoveBy(int x, int y);
}
```

```
public interface IShape<out T> : IShape
    where T : IShape<T>
{
    T Clone();
}
```

рекурсивные параметры-типы

прототип*

```
public class Circle(...)  
    : IShape<Circle>  
{  
    ...  
  
    public Circle Clone()  
    {  
        return new Circle(Position, Radius);  
    }  
}
```

рекурсивные параметры-типы

прототип*

```
T DuplicateShape<T>(T shape)
    where T : IShape<T>
{
    T copy = shape.Clone();
    copy.MoveBy(1, 1);

    return copy;
}
```

```
var circle = new Circle(new Position(1, 1), radius: 2);

Circle copiedCircle = DuplicateShape(circle);
copiedCircle.Stretch(1.2);
```